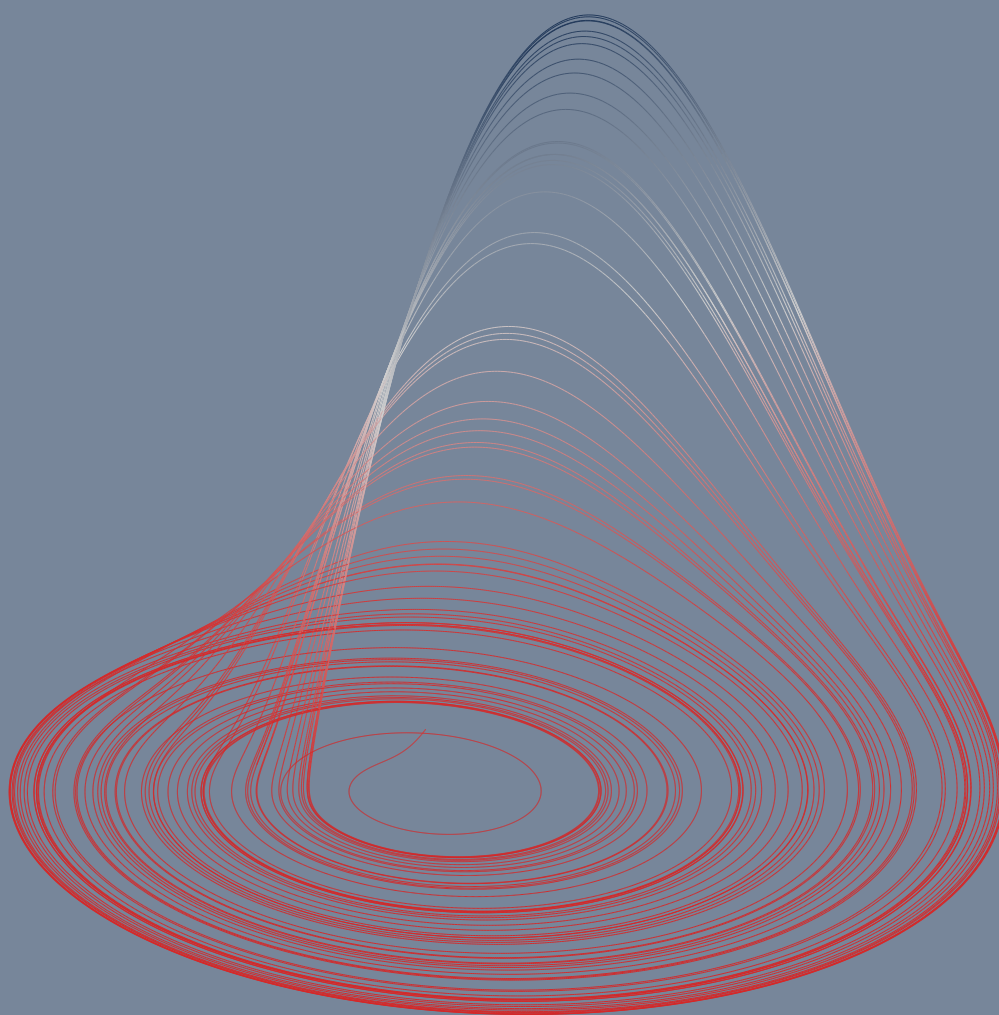


1^{ra} Edición

Métodos Espectrales y Variacionales

Implementación Computacional



Alejandro Soto González
Vicente Chavarría Gámez



Esta página fue dejada en blanco intencionalmente.

Métodos Espectrales y Variacionales

Implementación Computacional

Métodos Espectrales y Variacionales

Implementación Computacional

Alejandro
Soto González
ESFM-IPN

Vicente
Chavarría Gámez
ESFM-IPN



Contenido

Prefacio	III
Capítulo 1 Álgebra Lineal Numérica	2
1.1 Espacio Vectorial $\mathbb{R}^n$	3
1.2 Producto interno	6
1.3 Norma Euclidiana	8
1.4 Matrices sobre $\mathbb{R}$	9
1.5 Producto cruz	13
1.6 Determinante	14
1.7 Problemas	15
Capítulo 2 Ecuaciones Diferenciales Ordinarias	18
2.1 Fundamentos	19
2.2 Discretización y métodos numéricos	24
Bibliografía	B-I

Prefacio

Las Ecuaciones en Diferencias Parciales no son simplemente un conjunto de reglas y operadores; representan uno de los pilares más desafiantes y, a su vez, fascinantes de la matemática moderna. Son, en esencia, el lenguaje natural con el que el universo describe sus fenómenos más fundamentales. Su estudio exige una profunda capacidad de abstracción y un dominio riguroso del análisis matemático.

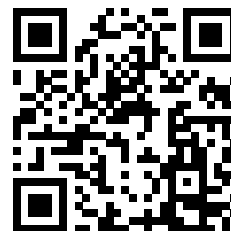
Este texto nace con el propósito de capturar la visión con la que el profesor **Alejandro Soto González**, matemático de formación, aborda esta cátedra en la Escuela Superior de Física y Matemáticas (ESFM). Desde su incorporación al cuerpo docente, el profesor Soto se ha distinguido por imprimir a sus clases un sello de rigor y estructura lógica que revitaliza la enseñanza de las matemáticas, ofreciendo a los estudiantes una perspectiva clara, moderna y profundamente fundamentada.

El material aquí presentado trasciende la categoría de una simple transcripción. Es el resultado de una meticulosa labor de recopilación y sistematización de las notas de clase tomadas por **Diana Guadalupe Ramírez Tovar**. Con una diligencia notable, ella logró documentar la esencia de estas lecciones, preservando el orden y la precisión que caracterizan la exposición del profesor.

Llevar estos apuntes manuscritos a un formato que haga justicia a su contenido requería un esfuerzo adicional de diseño y edición. Esta transformación hacia una obra tipográfica profesional ha sido realizada por **Vicente Chavarría Gámez**. A través del uso avanzado de $\text{\LaTeX}$, se ha buscado no solo la estética visual, sino la estructuración lógica que este nivel de matemáticas exige. Cada fórmula, cada definición e implementación algorítmica han sido codificadas para eliminar las barreras visuales entre el estudiante y el concepto.

Este volumen aspira a ser un testimonio del esfuerzo colaborativo y del alto nivel académico que se cultiva en nuestra institución. Se entrega a la comunidad docente y estudiantil como una herramienta sólida para aquellos que, al igual que nosotros, han decidido aceptar el reto de descifrar el mundo a través de las matemáticas.

Vicente Chavarría Gámez
México, 2026.



Dedicatoria

Para
Alejandro Soto González

Para mi madre
Vicente Chavarría Gámez

Capítulo 1

ÁLGEBRA LINEAL NUMÉRICA

Contenido

1.1	Espacio Vectorial $\mathbb{R}^n$	3
1.2	Producto interno . .	6
1.3	Norma Euclidiana . .	8
1.4	Matrices sobre $\mathbb{R}$. .	9
1.5	Producto cruz	13
1.6	Determinante	14
1.7	Problemas	15

Este capítulo presenta una revisión de los fundamentos del álgebra lineal sobre el cuerpo de los números reales, destacando por su enfoque dual que integra la teoría matemática con su implementación computacional. A lo largo del texto, se formalizan estructuras esenciales como el espacio vectorial $\mathbb{R}^n$ y el álgebra de matrices, abordando operaciones clave como lo son el producto interno, la norma euclidiana, el producto matricial, el producto cruz y el cálculo de determinantes. Alejándose de un tratamiento puramente teórico, cada sección acompaña las definiciones analíticas con algoritmos en pseudocódigo y su respectiva codificación en Python. Este enfoque integral tiene como objetivo proporcionar al lector tanto la base matemática sólida como las herramientas prácticas indispensables para la manipulación numérica requerida en el análisis de ecuaciones diferenciales parciales.

1.1 Espacio Vectorial $\mathbb{R}^n$

Uno de los pilares en el estudio del álgebra lineal es el estudio de los espacios vectoriales $\mathbb{R}^n$. Para comprender dichos espacios, primero debemos establecer las condiciones formales para que una estructura algebraica sea considerada un espacio vectorial.

Definición 1.1.1: Un **espacio vectorial** o **espacio lineal** V sobre un cuerpo $\mathbb{R}$ es un conjunto no vacío en el cual se definen dos operaciones: la suma de vectores $+: V \times V \rightarrow V$, y el producto por escalares $\cdot: \mathbb{R} \times V \rightarrow V$. Dichas operaciones deben satisfacer las siguientes propiedades para todos $u, v, w \in V$ y para todos $\lambda, \alpha \in \mathbb{R}$:

$S_1 \quad u + v \in V.$	$P_1 \quad \lambda \cdot u \in V.$
$S_2 \quad u + v = v + u.$	$P_2 \quad \lambda \cdot (\alpha \cdot u) = (\lambda\alpha) \cdot u.$
$S_3 \quad u + (v + w) = (u + v) + w.$	$P_3 \quad 1 \cdot u = u.$
$S_4 \quad \exists! 0 \in V, \quad u + 0 = 0 + u = u.$	$D_1 \quad \lambda \cdot (u + v) = \lambda \cdot u + \lambda \cdot v.$
$S_5 \quad \exists! (-u) \in V, \quad u + (-u) = 0.$	$D_2 \quad (\lambda + \alpha) \cdot u = \lambda \cdot u + \alpha \cdot u.$

En lo que resta del texto omitiremos el símbolo $\cdot$ cuando denotemos el producto escalar, i.e., $\lambda \cdot u \equiv \lambda u$. Ahora veremos otro concepto que resulta fundamental para desarrollar la teoría del álgebra lineal en la siguiente:

Definición 1.1.2 (n -tupla): Sea $n \in \mathbb{N}$, definimos el conjunto de n -tuplas de números reales como

$$\mathbb{R}^n := \{(x_1, x_2, \dots, x_n)^T \mid \forall j = 1, \dots, n, \quad x_j \in \mathbb{R}\}.$$

Sobre este conjunto definimos las siguientes operaciones:

$+: \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}^n,$	$\cdot: \mathbb{R} \times \mathbb{R}^n \rightarrow \mathbb{R}^n,$
$u = \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{pmatrix} \in \mathbb{R}^n, \quad v = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix} \in \mathbb{R}^n.$	$\lambda \in \mathbb{R}, \quad u = \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{pmatrix} \in \mathbb{R}^n.$
$u + v = \begin{pmatrix} u_1 + v_1 \\ u_2 + v_2 \\ \vdots \\ u_n + v_n \end{pmatrix}.$	$\lambda \cdot u = \begin{pmatrix} \lambda u_1 \\ \lambda u_2 \\ \vdots \\ \lambda u_n \end{pmatrix}.$

El siguiente ejemplo comprueba que $\mathbb{R}^3$ satisface las propiedades S_3 y P_2 de la Definición 1.1.1.

Ejemplo 1.1.1: Considere el conjunto $\mathbb{R}^3 := \{x = (x_1, x_2, x_3)^T \mid x \in \mathbb{R}^3 \wedge x_1, x_2, x_3 \in \mathbb{R}\}$ sobre el cual se definen las siguientes operaciones:

$+: \mathbb{R}^3 \times \mathbb{R}^3 \rightarrow \mathbb{R}^3.$	$\cdot: \mathbb{R} \times \mathbb{R}^3 \rightarrow \mathbb{R}^3.$
$u = \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} \in \mathbb{R}^3, \quad v = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} \in \mathbb{R}^3.$	$\lambda \in \mathbb{R}, \quad u = \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} \in \mathbb{R}^3.$
$u + v = \begin{pmatrix} u_1 + v_1 \\ u_2 + v_2 \\ u_3 + v_3 \end{pmatrix}.$	$\lambda \cdot u = \begin{pmatrix} \lambda u_1 \\ \lambda u_2 \\ \lambda u_3 \end{pmatrix}.$

Mostrar que el conjunto $(\mathbb{R}^3, +, \cdot)$ cumple todos los axiomas de un espacio vectorial.

Demostración. Los axiomas $S_1, S_3, S_4, S_5, P_1, P_3, D_1$ y D_2 se proponen como ejercicio para el lector. Veamos que el axioma S_2 se cumple. Sean $u = (u_1, u_2, u_3)^T, v = (v_1, v_2, v_3)^T \in \mathbb{R}^3$ vectores,

Espacio vectorial

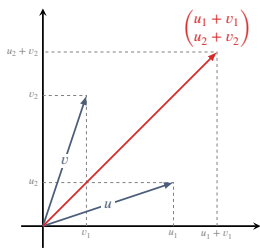


Figura 1.1: Representación geométrica de la suma de vectores en $\mathbb{R}^2$.

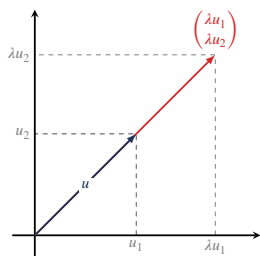


Figura 1.2: Representación geométrica del producto de un escalar por un vector en $\mathbb{R}^2$.

entonces

$$\begin{aligned}
 u + v &= \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} + \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} \\
 &= \begin{pmatrix} u_1 + v_1 \\ u_2 + v_2 \\ u_3 + v_3 \end{pmatrix} && \text{(por definición de la adición en } \mathbb{R}^3) \\
 &= \begin{pmatrix} v_1 + u_1 \\ v_2 + u_2 \\ v_3 + u_3 \end{pmatrix} && \text{(por conmutatividad de la adición en } \mathbb{R}) \\
 &= \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} + \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} && \text{(por definición)} \\
 &= v + u.
 \end{aligned}$$

Ahora, veamos que el axioma P_2 se cumple. Sea $u = (u_1, u_2, u_3)^T \in \mathbb{R}^3$ un vector y sean $\lambda, \alpha \in \mathbb{R}$ escalares. Entonces

$$\begin{aligned}
 \lambda(\alpha u) &= \lambda \left(\alpha \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} \right) \\
 &= \lambda \begin{pmatrix} \alpha u_1 \\ \alpha u_2 \\ \alpha u_3 \end{pmatrix} && \text{(por definición del producto por escalares en } \mathbb{R}^3) \\
 &= \begin{pmatrix} (\lambda \alpha) u_1 \\ (\lambda \alpha) u_2 \\ (\lambda \alpha) u_3 \end{pmatrix} && \text{(por definición y por asociatividad del producto en } \mathbb{R}) \\
 &= (\lambda \alpha) \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} && \text{(por definición)} \\
 &= (\lambda \alpha) u.
 \end{aligned}$$

En conclusión, el conjunto $(\mathbb{R}^3, +, \cdot)$ cumple con todos los axiomas de un espacio vectorial, por lo tanto, $\mathbb{R}^3$ es un espacio vectorial. ■

Tras demostrar que $(\mathbb{R}^3, +, \cdot)$ satisface los axiomas de un espacio vectorial, el siguiente paso es traducir estas operaciones a un lenguaje algorítmico. En la práctica, estas operaciones se ejecutan componente a componente, lo que nos permite generalizar la suma para cualquier vector en $\mathbb{R}^n$ mediante el siguiente algoritmo:

Algoritmo

1.1.1

Suma vectorial

Entrada: $u, v \in \mathbb{R}^n$ (*Deben tener la misma dimensión*)

Salida: Vector suma $s \in \mathbb{R}^n$

función SUMA VECTORIAL(u, v)

$n \leftarrow \text{dimensión}(u)$

$s \leftarrow \underbrace{[0, 0, \dots, 0]}_{n\text{-veces}}$ (*Inicializar vector de n-ceros*)

para $1 \leq j \leq n$ **hacer**

$s_j \leftarrow s_j + (u_j + v_j)$ (*Suma componente a componente*)

fin para

retornar s

fin función

Con la lógica del algoritmo establecida en el Algoritmo 1.1.1, su implementación en Python resulta

inmediata:

Código 1.1.1 | Implementación de la Suma Vectorial en Python

```

1 def suma_vectores(u, v):
2     # 1. Validación de dimensiones
3     # Matemáticamente, u y v deben pertenecer al mismo espacio V
4     if len(u) != len(v):
5         raise ValueError("Los vectores deben tener la misma dimensión.")
6
7     n = len(u)
8
9     # 2. Inicialización del vector resultado
10    # Creamos una lista con 'n' ceros
11    s = [0] * n
12
13    # 3. Iteración y suma componente a componente
14    # range(n) genera índices desde 0 hasta n-1
15    for j in range(n):
16        s[j] = u[j] + v[j]
17
18    return s

```

A continuación, abordamos el **producto por un escalar**. Dado que esta operación también actúa componente a componente, podemos aplicar una estructura iterativa análoga a la utilizada en el Algoritmo 1.1.1:

Producto por escalares

Entrada: u un vector en $\mathbb{R}^n$, λ un escalar en $\mathbb{R}$

Salida: Vector escalado $p \in \mathbb{R}^n$

función PRODUCTO POR ESCALARES(λ, u)

```

     $n \leftarrow \text{dimensión}(u)$ 
     $p \leftarrow [0, 0, \dots, 0]$  (Inicializar vector de n-ceros)
     $\underbrace{\hspace{1.5cm}}_{n\text{-veces}}$ 

    para  $1 \leq j \leq n$  hacer
    |    $p_j \leftarrow p_j + (\lambda \cdot u_j)$  (Multipliación componente a componente)
    fin para
    retornar  $p$ 
fin función

```

Algoritmo

1.1.2

De acuerdo con el algoritmo anterior, el código de Python queda de la siguiente forma:

Código 1.1.2 | Implementación del Producto por escalares en Python.

```

1 import numpy as np
2
3 def producto_escalar(a, u):
4     # 1. Obtener la dimensión del espacio vectorial
5     # El vector 'u' pertenece a  $\mathbb{R}^n$ 
6     n = len(u)
7
8     # 2. Inicialización del vector resultado
9     # Creamos una lista con 'n' ceros
10    p = np.zeros(n)
11
12    # 3. Iteración y producto componente a componente
13    # Aplicamos la definición:  $(a * u)_j = a * u_j$  para todo  $j$ 
14    for j in range(n):

```

```

15     p[j] = a * u[j]
16
17     return p

```

De esta forma quedan establecidos los fundamentos tanto teóricos como computacionales, lo cual nos permite introducir el siguiente concepto.

1.2 Producto interno

Como mencionamos anteriormente, los espacios vectoriales son un pilar del álgebra lineal; sin embargo, esta estructura se enriquece al definir el producto interno. Este concepto dota al espacio vectorial de herramientas geométricas, permitiendo medir longitudes y ángulos. Recordemos que todo espacio con producto interno es, por definición, un espacio vectorial. Dicho esto, pasemos a ver la siguiente:

Producto interno canónico

Definición 1.2.1: Sean $u, v \in \mathbb{R}^n$ tales que $u = [u_i]_{i=1}^n$ y $v = [v_i]_{i=1}^n$. Entonces el **producto interno canónico** de u con v tal que $\langle \cdot, \cdot \rangle : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ se define como:

$$\langle u, v \rangle = \sum_{i=1}^n u_i \cdot v_i = u_1 v_1 + u_2 v_2 + u_3 v_3 + \cdots + u_n v_n.$$

Además, el producto interno satisface las siguientes propiedades para todo $u, v, w \in \mathbb{R}^n$ y para todo $\lambda \in \mathbb{R}$:

- i) $\forall u, v \in \mathbb{R}^n, \quad \langle u, v \rangle = \langle v, u \rangle.$
- ii) $\forall u, v \in \mathbb{R}^n, \forall \lambda \in \mathbb{R}, \quad \langle \lambda u, v \rangle = \lambda \langle u, v \rangle = \langle u, \lambda v \rangle$
- iii) $\forall u, v, w \in \mathbb{R}^n, \quad \langle u + v, w \rangle = \langle u, w \rangle + \langle v, w \rangle.$
- iv) $\langle u, u \rangle \geq 0$ y es cero si y solo si $u = 0_V$.

Para ilustrar estas propiedades, se verifica el cumplimiento de estos axiomas para el espacio $\mathbb{R}^3$:

Ejemplo 1.2.1: Sean $u = [u_i]_{i=1}^3, v = [v_i]_{i=1}^3 \in \mathbb{R}^3$. Se define el producto interno como:

$$\langle u, v \rangle = u_1 v_1 + u_2 v_2 + u_3 v_3.$$

Mostrar que el producto interno satisface los axiomas.

Demostración. Verifiquemos las propiedades para $u, v, w \in \mathbb{R}^3$ y $\lambda \in \mathbb{R}$ definidos como:

$$u = \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix}, \quad v = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} \quad \text{y} \quad w = \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix}.$$

$$i) \quad \forall u, v \in \mathbb{R}^3, \quad \langle u, v \rangle = \langle v, u \rangle$$

$$\begin{aligned} \langle u, v \rangle &= u_1 v_1 + u_2 v_2 + u_3 v_3 && \text{(por definición del producto interno)} \\ &= v_1 u_1 + v_2 u_2 + v_3 u_3 && \text{(por conmutatividad del producto en } \mathbb{R} \text{)} \\ &= \langle v, u \rangle. && \text{(por definición)} \end{aligned}$$

Por tanto, el producto interno definido es **simétrico**.

$$ii) \quad \forall u, v \in \mathbb{R}^3, \forall \lambda \in \mathbb{R}, \quad \langle \lambda u, v \rangle = \lambda \langle u, v \rangle = \langle u, \lambda v \rangle$$

$$\begin{aligned} \langle \lambda u, v \rangle &= (\lambda u_1) v_1 + (\lambda u_2) v_2 + (\lambda u_3) v_3 && \text{(por definición)} \\ &= \lambda (u_1 v_1) + \lambda (u_2 v_2) + \lambda (u_3 v_3) && \text{(por asociatividad del producto en } \mathbb{R} \text{)} \\ &= \lambda \langle u, v \rangle. && \text{(por definición)} \end{aligned}$$

Por tanto, el producto interno definido es **homogéneo**.

$$\text{iii)} \quad \forall u, v, w \in \mathbb{R}^3, \quad \langle u + v, w \rangle = \langle u, w \rangle + \langle v, w \rangle$$

$$\begin{aligned} \langle u + v, w \rangle &= (u_1 + v_1)w_1 + (u_2 + v_2)w_2 + (u_3 + v_3)w_3 && \text{(por definición)} \\ &= u_1w_1 + v_1w_1 + u_2w_2 + v_2w_2 + u_3w_3 + v_3w_3 && \text{(por distributividad en } \mathbb{R} \text{)} \end{aligned}$$

Agrupando de forma conveniente, tenemos

$$\begin{aligned} &= (u_1w_1 + u_2w_2 + u_3w_3) + (v_1w_1 + v_2w_2 + v_3w_3) \\ &= \langle u, w \rangle + \langle v, w \rangle. && \text{(por definición)} \end{aligned}$$

Por tanto, el producto interno definido es **distributivo**.

$$\text{iv)} \quad \langle u, u \rangle \geq 0 \text{ y es cero si y solo si } u = 0_{\mathbb{R}^3}$$

$$\begin{aligned} \langle u, u \rangle &= u_1u_1 + u_2u_2 + u_3u_3 && \text{(por definición)} \\ &= u_1^2 + u_2^2 + u_3^2. \end{aligned}$$

Como $x^2 \geq 0$ para todo $x \in \mathbb{R}$, la suma anterior es **no negativa**, por lo que $\langle u, u \rangle \geq 0$.

Además, si $\langle u, u \rangle = 0$, entonces

$$u_1^2 + u_2^2 + u_3^2 = 0.$$

Dado que es una suma de términos no negativos, esto implica necesariamente que $u_1 = 0, u_2 = 0, u_3 = 0$. Por lo tanto, $u = 0_{\mathbb{R}^3}$.

Como la operación propuesta satisface todas las propiedades axiomáticas, concluimos que es un producto interno. En particular, se trata del producto interno canónico de $\mathbb{R}^3$. ■

Con los fundamentos teóricos claros, el siguiente paso es la implementación. El algoritmo que se muestra a continuación generaliza el cálculo de la suma de productos para vectores de cualquier tamaño.

Producto interno

Entrada: $u, v \in \mathbb{R}^n$ (*Deben tener la misma dimensión*)

Salida: Producto interno $\pi \in \mathbb{R}$

función PRODUCTO_INTERNO(u, v)

$n \leftarrow \text{dimensión}(u)$

$\pi \leftarrow 0$ (*Inicializar la suma en cero*)

para $1 \leq j \leq n$ **hacer**

$\pi \leftarrow \pi + (u_j \cdot v_j)$ (*Acumula la suma de productos*)

fin para

retornar π

fin función

Algoritmo 1.2.1

Basándonos en el diseño lógico anterior, la implementación en Python sigue una estructura idéntica. Se incluye, además, una validación explícita de las dimensiones para garantizar una mayor robustez del código:

Código 1.2.1 | Implementación del Producto interno en Python.

```
1 def inner_prod(u, v):
2     # 1. Validación de compatibilidad dimensional
3     # Matemáticamente, u y v deben pertenecer al mismo espacio V
4     if len(u) != len(v):
5         raise ValueError("Los vectores deben tener la misma dimensión")
6
7     # 2. Obtener la dimensión del espacio vectorial
8     # Definimos n como la cardinalidad de la base del espacio
9     n = len(u)
10
11     # 3. Inicialización del acumulador (Escalar)
```

```

12     # Inicializamos la sumatoria en 0
13     ip = 0
14
15     # 4. Iteración y cálculo de la sumatoria
16     # Implementamos la definición: <u,v> = sum(u_j * v_j) para todo j
17     for j in range(n):
18         ip += u[j] * v[j]
19
20     return ip

```

Habiendo definido e implementado el producto interno, tenemos la herramienta matemática necesaria para calcular la “longitud” o magnitud de nuestros vectores. Esto nos lleva de forma natural al siguiente concepto.

1.3 Norma Euclidiana

A esta noción de “tamaño” o “longitud” de un vector se le conoce formalmente en álgebra lineal como su norma. Gracias a que ya contamos con el producto interno, su definición matemática es bastante directa:

Norma

Definición 1.3.1: Sea $u = [u_j]_{j=1}^n \in \mathbb{R}^n$. La **norma** de u se define como

$$\|u\| = \sqrt{\langle u, u \rangle} = \sqrt{u_1^2 + u_2^2 + u_3^2 + \cdots + u_n^2}.$$

Para la implementación de la definición anterior, requerimos calcular la raíz cuadrada del producto interno del vector consigo mismo. La lógica computacional se resume a continuación:

Algoritmo

1.3.1

Norma euclidiana

Entrada: $u \in \mathbb{R}^n$

Salida: La norma euclidiana del vector u ($\|u\|$)

función NORMA EUCLIDIANA(u)

$pi \leftarrow \text{PRODUCTO INTERNO}(u, u)$

$norma \leftarrow \sqrt{pi}$

retornar norma

fin función

Ejercicio 1.3.1 (Implementación de la Norma en Python): El sistema nativo de Python posee la función `sqrt(...)` dentro de la librería `math` para manejar raíces cuadradas. Utilizando esto y apoyándose en el Código 1.2.1, programe una función que reciba un vector y retorne su norma.



Nota:

Debido a la precisión de punto flotante en lenguajes como Python, operaciones que matemáticamente resultan en cero a menudo devuelven un valor minúsculo (del orden de 10^{-14} a 10^{-16}). Es a este **cero computacional** o tolerancia al que nos referimos en la práctica.

Ya que dominamos el cálculo de la magnitud de un vector individual, podemos utilizar nuevamente el producto interno, pero esta vez para analizar la relación geométrica entre dos vectores distintos.

Un caso de vital importancia en el álgebra lineal ocurre cuando esta operación se anula. Geométricamente, si el producto interno de dos vectores distintos de cero es igual a cero, significa que el ángulo entre ellos es de 90 grados; es decir, son ortogonales (perpendiculares) entre sí.

Definición 1.3.2: Sean $u, v \in \mathbb{R}^n$. Decimos que u es ortogonal a v si $\langle u, v \rangle = 0$.

Dado que el rigor del “cero absoluto” es difícil de alcanzar en la programación debido a errores de redondeo, el diseño de nuestro algoritmo debe incorporar una tolerancia para absorber estas pequeñas imprecisiones numéricas:

Algoritmo

1.3.2

Ortogonalidad de dos vectores

Entrada: $u, v \in \mathbb{R}^n$ (*Deben tener la misma dimensión*)

Salida: Son ortogonales o no son ortogonales

```

función ORTOGONALIDAD( $u, v$ )
     $pi \leftarrow$  PRODUCTO INTERNO( $u, v$ )
     $TOL \leftarrow 10^{-14}$  (Así contrarrestamos los errores de redondeo)
    si  $|pi| \leq TOL$  entonces
        | retornar “Son ortogonales.”
    si no
        | retornar “No son ortogonales.”
    fin si
fin función

```

Ejercicio 1.3.2 (Verificación de Ortogonalidad en Python): Llevando esta lógica computacional a Python, escriba una función que reciba dos vectores y determine si son ortogonales. Utilice la función nativa `abs(...)` para evaluar el valor absoluto del producto interno (apoyándose nuevamente en el Código 1.2.1) y compárelo contra una tolerancia predeterminada de 10^{-14} . Se recomienda que su función retorne un valor lógico (True o False).

Con la verificación de la ortogonalidad concluimos nuestro estudio fundamental sobre vectores individuales y sus operaciones básicas. El siguiente paso natural en nuestro nivel de abstracción es extender estas ideas a estructuras bidimensionales. Al agrupar escalares en “arreglos rectangulares”, entramos al estudio de las matrices, las cuales nos permitirán representar transformaciones y sistemas de ecuaciones de manera compacta.

1.4 Matrices sobre $\mathbb{R}$

De manera intuitiva, solemos visualizar una matriz sobre el campo de los números reales $\mathbb{R}$ como un arreglo de escalares ordenados en m filas y n columnas. Sin embargo, para construir una base analítica rigurosa, estableceremos su definición en términos funcionales.

Definición 1.4.1: Sean $m, n \in \mathbb{N}$. Una matriz A de orden $m \times n$ sobre $\mathbb{R}$ se define formalmente como una función $A : \{1, \dots, m\} \times \{1, \dots, n\} \rightarrow \mathbb{R}$.

Convencionalmente, la imagen del par ordenado (i, j) se denota por $a_{i,j} := A_{i,j}$, la cual representa el escalar ubicado en la i -ésima fila y la j -ésima columna. Su representación tabular equivalente es:

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{pmatrix}.$$

Al conjunto de todas las matrices de orden $m \times n$ con entradas reales se le denota como $\mathcal{M}_{m \times n}(\mathbb{R})$, cumpliendo que $A \in \mathcal{M}_{m \times n}(\mathbb{R})$ si y solo si $a_{i,j} \in \mathbb{R}$ para cada $i \in \{1, \dots, m\}$ y $j \in \{1, \dots, n\}$.

Para los propósitos de este texto, restringiremos nuestro estudio analítico y computacional al conjunto de las **matrices cuadradas**, es decir, aquellas donde el número de filas es igual al número de columnas ($m = n$), las cuales denotaremos de forma compacta como $\mathcal{M}_{n \times n}(\mathbb{R})$ o equivalentemente $\mathcal{M}_n(\mathbb{R})$.

Además, con el fin de ilustrar las operaciones algebraicas de forma clara, utilizaremos ejemplos en $\mathcal{M}_2(\mathbb{R})$, teniendo en cuenta que su generalización a dimensiones mayores es analíticamente inmediata y se verá reflejada de forma directa en los algoritmos computacionales propuestos.

1.4.1 Suma de matrices

Para que la suma de dos matrices esté definida, ambas deben tener exactamente las mismas dimensiones. Siguiendo nuestro enfoque, ilustraremos y definiremos esta operación para el caso de matrices cuadradas bidimensionales.

Definición 1.4.2: Sean $A, B \in \mathcal{M}_{2 \times 2}(\mathbb{R})$ definidas como:

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{pmatrix}, \quad B = \begin{pmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \end{pmatrix}.$$

La **suma de matrices**, denotada por $A + B$, se define como la matriz resultante al sumar los elementos correspondientes de cada matriz (entrada por entrada):

$$A + B = \begin{pmatrix} a_{1,1} + b_{1,1} & a_{1,2} + b_{1,2} \\ a_{2,1} + b_{2,1} & a_{2,2} + b_{2,2} \end{pmatrix}.$$

Esta misma lógica aplica para cualquier par de matrices en $\mathcal{M}_{n \times n}(\mathbb{R})$. Dado que el proceso consiste en operar sistemáticamente sobre cada posición de la cuadrícula, su generalización computacional resulta muy directa, tal como se detalla en el siguiente algoritmo:

Algoritmo

1.4.1

Suma Matricial

Entrada: $A, B \in \mathcal{M}_{n \times n}$

Salida: Matriz suma $S \in \mathcal{M}_{n \times n}$

```

función SUMA DE MATRICES( $A, B$ )
     $n \leftarrow \text{dimensión}(A)$  (Al ser cuadrada, filas = columnas = n)
     $S \leftarrow \text{matriz\_ceros}(n, n)$ 
    para  $1 \leq i \leq n$  hacer (Recorre las filas)
        para  $1 \leq j \leq n$  hacer (Recorre las columnas)
             $S_{i,j} \leftarrow S_{i,j} + (A_{i,j} + B_{i,j})$ 
        fin para
    fin para
    retornar  $S$ 
fin función

```

Ejercicio 1.4.1 (Implementación de la suma matricial en Python): Basándose en la lógica iterativa descrita en el algoritmo anterior, desarrolle un script en Python que implemente la suma de dos matrices cuadradas de dimensión n .

Habiendo establecido el mecanismo para combinar dos matrices de las mismas dimensiones, el siguiente paso lógico en la construcción de nuestra álgebra matricial es analizar cómo interactúa una matriz con un elemento individual del campo $\mathbb{R}$.

1.4.2 Producto de un escalar por una matriz

La multiplicación de una matriz por un escalar es una operación directa donde el valor real amplifica o contrae cada una de las entradas de la matriz de manera uniforme.

Definición 1.4.3: Sean una matriz $A \in \mathcal{M}_{2 \times 2}(\mathbb{R})$ y un escalar $\lambda \in \mathbb{R}$, definidos como:

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{pmatrix}.$$

El **producto de un escalar por una matriz**, denotado por λA , se define multiplicando cada entrada de A por el escalar λ :

$$\lambda A = \begin{pmatrix} \lambda a_{1,1} & \lambda a_{1,2} \\ \lambda a_{2,1} & \lambda a_{2,2} \end{pmatrix}.$$

Ejercicio 1.4.2 (Implementación computacional del producto de un escalar por una matriz): Desarrolle el algoritmo computacional en pseudocódigo que reciba como parámetros una matriz cuadrada de dimensión n y un escalar λ , retornando la matriz resultante λA . Posteriormente, escriba el código para implementar este algoritmo en Python.

Hasta este punto, hemos definido operaciones elementales que preservan la naturaleza estructural

del espacio matricial. El siguiente nivel de interacción consiste en vincular nuestras matrices con los vectores columna que definimos al inicio de este texto. Esta nueva operación representa un salto conceptual importante, ya que sienta las bases para interpretar a las matrices como transformaciones lineales que actúan sobre los elementos de un espacio vectorial.

1.4.3 Producto de una matriz por un vector

El producto de una matriz por un vector columna produce un nuevo vector columna. A diferencia de las operaciones anteriores, esta no se realiza simplemente entrada por homóloga, sino que cada componente del vector resultante se calcula sumando el producto de cada elemento de la fila de la matriz por el elemento correspondiente del vector.

Definición 1.4.4: Sean una matriz $A \in \mathcal{M}_{2 \times 2}(\mathbb{R})$ y un vector columna $v \in \mathbb{R}^2$, definidos como:

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{pmatrix}, \quad v = \begin{pmatrix} v_1 \\ v_2 \end{pmatrix}.$$

El **producto de la matriz por el vector**, denotado como Av , resulta en un nuevo vector en $\mathbb{R}^2$ definido como:

$$Av = \begin{pmatrix} a_{1,1}v_1 + a_{1,2}v_2 \\ a_{2,1}v_1 + a_{2,2}v_2 \end{pmatrix}.$$

Ejercicio 1.4.3 (Implementación computacional del producto de una matriz con un vector): Basándose en la definición mecánica de la dimensión 2×2 , plantee el algoritmo en pseudocódigo generalizado para calcular el producto de una matriz $A \in \mathcal{M}_{n \times n}(\mathbb{R})$ por un vector $v \in \mathbb{R}^n$. Implemente la solución en código Python para verificar la validez de su lógica iterativa.

A diferencia de la suma o el producto por un escalar, la multiplicación entre dos matrices no se realiza entrada por entrada. Esta operación tiene sus raíces en la composición de transformaciones lineales, lo que analíticamente se traduce en el cálculo de productos internos entre las filas de la primera matriz y las columnas de la segunda.

1.4.4 Producto de matrices

Habiendo definido el producto de una matriz por un vector, resulta natural preguntarnos si es posible extender esta operación para multiplicar dos matrices entre sí. La respuesta es afirmativa. En lugar de ser una simple multiplicación entrada por entrada, el producto matricial generaliza la idea de aplicar una matriz a múltiples vectores de manera simultánea, entrelazando las filas de la primera con las columnas de la segunda.

Definición 1.4.5: Sean $A, B \in \mathcal{M}_{2 \times 2}(\mathbb{R})$ dos matrices definidas por sus componentes como:

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{pmatrix}, \quad B = \begin{pmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \end{pmatrix}.$$

El **producto matricial**, denotado como $A \cdot B$ (o simplemente AB), resulta en una nueva matriz en $\mathcal{M}_{2 \times 2}(\mathbb{R})$ definida como:

$$AB = \begin{pmatrix} a_{1,1}b_{1,1} + a_{1,2}b_{2,1} & a_{1,1}b_{1,2} + a_{1,2}b_{2,2} \\ a_{2,1}b_{1,1} + a_{2,2}b_{2,1} & a_{2,1}b_{1,2} + a_{2,2}b_{2,2} \end{pmatrix}.$$

De forma más intuitiva, el producto de matrices se realiza como sigue:

$$A \cdot B = \begin{pmatrix} \boxed{a_{1,1} \ a_{1,2}} \\ a_{2,1} \ a_{2,2} \end{pmatrix} \cdot \begin{pmatrix} \boxed{b_{1,1}} \ b_{1,2} \\ b_{2,1} \ b_{2,2} \end{pmatrix} = \begin{pmatrix} \boxed{a_{1,1}b_{1,1} + a_{1,2}b_{2,1}} & a_{1,1}b_{1,2} + a_{1,2}b_{2,2} \\ a_{2,1}b_{1,1} + a_{2,2}b_{2,1} & a_{2,1}b_{1,2} + a_{2,2}b_{2,2} \end{pmatrix}.$$

Dando paso a la clásica interpretación de “filas por columnas”.

Desde una perspectiva computacional, la sumatoria explícita en la definición analítica requiere la implementación de un tercer ciclo anidado. El siguiente algoritmo detalla el método iterativo clásico para calcular el producto de matrices cuadradas de dimensión n :

Algoritmo

1.4.2

Producto matricial tradicional

Entrada: $A, B \in \mathcal{M}_{n \times n}$

Salida: Matriz producto $P \in \mathcal{M}_{n \times n}$

función PRODUCTO MATRICIAL(A, B)

$n \leftarrow \text{dimensión}(A)$

$P \leftarrow \text{matriz.ceros}(n, n)$

para $1 \leq i \leq n$ **hacer** *(Recorre las filas de la matriz A)*

para $1 \leq j \leq n$ **hacer** *(Recorre las columnas de la matriz B)*

para $1 \leq k \leq n$ **hacer** *(Efectúa la suma de productos cruzados)*

$P_{i,j} \leftarrow P_{i,j} + (A_{i,k} \cdot B_{k,j})$

fin para

fin para

fin para

retornar P

fin función

Si observamos con detenimiento el ciclo más profundo (la variable k), notaremos que la operación que realiza es exactamente el cálculo del producto interno entre dos vectores. Aprovechando este hecho y adoptando un enfoque de programación modular, podemos abstraer el ciclo interno delegándolo a una función auxiliar de producto interno, resultando en un algoritmo mucho más legible y elegante:

Algoritmo

1.4.3

Producto matricial mediante producto interno

Entrada: $A, B \in \mathcal{M}_{n \times n}$

Salida: Matriz producto $P \in \mathcal{M}_{n \times n}$

función PRODUCTO MATRICIAL(A, B)

$n \leftarrow \text{dimensión}(A)$

$P \leftarrow \text{matriz.ceros}(n, n)$

para $1 \leq i \leq n$ **hacer** *(Recorre las filas de la matriz A)*

para $1 \leq j \leq n$ **hacer** *(Recorre las columnas de la matriz B)*

$P_{i,j} \leftarrow \text{PRODUCTO INTERNO}(A.\text{fila}(i), B.\text{columna}(j))$

fin para

fin para

retornar P

fin función

Ejercicio 1.4.4 (Implementación y análisis del producto matricial en Python.): Basándose en los dos algoritmos presentados en esta sección, desarrolle un script en Python que implemente y compare ambas metodologías para la multiplicación de matrices cuadradas. Siga las siguientes directrices:

- 1. Implementación iterativa:** Defina la función `producto_matricial_tradicional(A, B)` utilizando exclusivamente tres ciclos `for` anidados, respetando la estructura del primer algoritmo.
- 2. Implementación modular:** Defina primero una función auxiliar `producto_interno(v, u)` que calcule el producto punto entre dos vectores (listas unidimensionales). Posteriormente, construya `producto_matricial_modular(A, B)` utilizando esta función auxiliar dentro de un doble ciclo `for`.
- 3. Validación de exactitud:** Genere dos matrices aleatorias en $\mathcal{M}_3(\mathbb{R})$. Compruebe que ambas funciones devuelven el mismo resultado y verifique su exactitud comparándolas con el operador nativo de multiplicación matricial de NumPy (`A @ B`).
- 4. Análisis de rendimiento:** Utilice el módulo `time` de Python para medir el tiempo de eje-

cución de ambas funciones. Pruebe su código con matrices de dimensiones crecientes (por ejemplo, $n = 50, 100, 200$). ¿Observa alguna diferencia significativa en el tiempo de procesamiento al abstraer el tercer ciclo en una función auxiliar? Documente sus conclusiones.

1.5 Producto cruz

A diferencia del producto interno, el producto cruz es una operación exclusiva del espacio tridimensional que toma dos vectores y produce un tercer vector espacial. Analíticamente, podemos visualizarlo como el mapeo:

$$f(\vec{a}, \vec{b}) : \mathbb{R}^3 \times \mathbb{R}^3 \xrightarrow{\vec{a} \times \vec{b}} \mathbb{R}^3.$$

Propiedades geométricas

- **Ortogonalidad:** El vector resultante $\vec{c} = \vec{a} \times \vec{b}$ es ortogonal al paralelogramo formado por $\vec{a}$ y $\vec{b}$. Es decir, $\vec{a} \bullet \vec{c} = 0$ y $\vec{b} \bullet \vec{c} = 0$.
- **Magnitud:** La norma del vector resultante, denotada como $\|\vec{c}\|$, equivale matemáticamente al área del paralelogramo formado por los vectores $\vec{a}$ y $\vec{b}$. Se calcula mediante la relación:

$$\|\vec{c}\| = \|\vec{a}\| \|\vec{b}\| \sin \theta$$

donde θ es el ángulo menor comprendido entre ambos vectores.

Definición 1.5.1: Sean $\vec{a}, \vec{b} \in \mathbb{R}^3$ expresados en la base canónica $\{\hat{i}, \hat{j}, \hat{k}\}$. El **producto cruz** se define mediante el desarrollo formal del siguiente determinante por cofactores:

$$\begin{aligned} \vec{a} \times \vec{b} &= \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix} = \begin{vmatrix} a_2 & a_3 \\ b_2 & b_3 \end{vmatrix} \hat{i} - \begin{vmatrix} a_1 & a_3 \\ b_1 & b_3 \end{vmatrix} \hat{j} + \begin{vmatrix} a_1 & a_2 \\ b_1 & b_2 \end{vmatrix} \hat{k} \\ &= (a_2 b_3 - a_3 b_2) \hat{i} + (a_3 b_1 - a_1 b_3) \hat{j} + (a_1 b_2 - a_2 b_1) \hat{k}. \end{aligned}$$

Dado que su definición analítica es puramente algebraica, la evaluación computacional del producto cruz es directa mediante el método de cofactores, tal como se detalla a continuación:

Producto cruz

Entrada: $\vec{a}, \vec{b} \in \mathbb{R}^3$

Salida: Producto cruz $\vec{c} \in \mathbb{R}^3$

función PRODUCTO CRUZ($\vec{a}, \vec{b}$)

(Cálculo de componentes espaciales)

$c_1 \leftarrow a_2 b_3 - a_3 b_2$

$c_2 \leftarrow a_3 b_1 - a_1 b_3$

$c_3 \leftarrow a_1 b_2 - a_2 b_1$

retornar $\vec{c} = (c_1, c_2, c_3)$

fin función

Algoritmo

1.5.1

Ejercicio 1.5.1 (Implementación y verificación del producto cruz): Desarrolle una función en Python que calcule el producto cruz entre dos vectores de $\mathbb{R}^3$ basándose en el algoritmo anterior. Una vez obtenida la implementación, realice las siguientes tareas de verificación:

1. **Cálculo de prueba:** Defina los vectores $\vec{a} = (1, 2, 3)$ y $\vec{b} = (4, 5, 6)$. Calcule manualmente el resultado y verifique que su función devuelve el vector esperado $\vec{c} = (-3, 6, -3)$.
2. **Comprobación de ortogonalidad:** Utilizando la función de producto interno desarrollada anteriormente, demuestre computacionalmente que el vector resultante $\vec{c}$ es perpendicular a los vectores originales. Es decir, verifique que:

$$\vec{a} \bullet \vec{c} \approx 0 \quad \text{y} \quad \vec{b} \bullet \vec{c} \approx 0.$$

Nota:

En mecánica y física clásica, el símbolo $\bullet$ o $\cdot$ denota el producto interno canónico.

Producto cruz

3. Análisis de precisión: Debido a la naturaleza de la aritmética de punto flotante en computación, el resultado del producto punto puede no ser exactamente 0.0, sino un valor muy pequeño (por ejemplo, 1.2×10^{-16}). Implemente una lógica de validación que utilice una tolerancia ($\epsilon = 10^{-14}$) para confirmar que el resultado es, para fines prácticos, cero.

Resulta evidente que la definición operativa del producto cruz depende intrínsecamente de un arreglo particular de componentes que hemos resuelto mediante el uso de determinantes. Sin embargo, esta herramienta no es exclusiva del espacio tridimensional ni de la geometría de vectores. Para consolidar nuestra capacidad de análisis sobre sistemas lineales y transformaciones, es imperativo formalizar el estudio del **determinante** como un escalar fundamental que encapsula propiedades globales de cualquier matriz cuadrada, tales como su invertibilidad y el factor de escala que impone sobre el espacio.

1.6 Determinante

El determinante es una función escalar que asigna a cada matriz cuadrada un valor real único, el cual proporciona información crítica sobre la invertibilidad de la matriz y el factor de escala de la transformación lineal asociada. Formalmente, lo definimos como el mapeo:

$$\det(A) : \mathcal{M}_n(\mathbb{R}) \rightarrow \mathbb{R}.$$

Propiedades axiomáticas

Para que una función sea considerada un determinante, debe satisfacer tres axiomas fundamentales:

- **Multilinealidad:** Es una función lineal respecto a cada fila (o columna) individualmente.
- **Alternancia:** Si dos filas (o columnas) son idénticas, el determinante es cero.
- **Normalización:** El determinante de la matriz identidad es la unidad, $\det(I) = 1$.

A partir de estos axiomas, se derivan propiedades útiles para el análisis matricial:

1. Si una matriz posee una fila o columna compuesta íntegramente por ceros, entonces $\det(A) = 0$.
2. Si una fila (o columna) es múltiplo escalar de otra, entonces $\det(A) = 0$.
3. El intercambio de dos filas (o columnas) invierte el signo del determinante.
4. Si se multiplica una sola fila (o columna) por un escalar k , el determinante resultante queda multiplicado por k .
5. El determinante del producto de dos matrices es el producto de sus determinantes: $\det(AB) = \det(A)\det(B)$.
6. Una matriz A es invertible si y solo si $\det(A) \neq 0$.

Definición 1.6.1 (Expansión de Laplace): Sea $A \in \mathcal{M}_{n \times n}(\mathbb{R})$ una matriz cuadrada. La **expansión de Laplace** (o expansión por cofactores) permite calcular el determinante de forma recursiva. Para cualquier fila i , se define como:

$$\det(A) = \sum_{j=1}^n (-1)^{i+j} a_{i,j} \det(A_{i,j})$$

donde $A_{i,j}$ denota la submatriz de orden $(n-1) \times (n-1)$ obtenida al eliminar la fila i y la columna j de la matriz original.

Desde el punto de vista algorítmico, la expansión de Laplace es una técnica recursiva de “divide y vencerás”. El siguiente algoritmo describe este proceso, utilizando las dimensiones 1×1 y 2×2 como casos base:

Nota:

Existen dos enfoques para realizar cálculos: el simbólico y el numérico. Para los fines computacionales de esta obra, priorizaremos el **cálculo numérico**. Este enfoque es más conveniente porque nos permite detener procesos iterativos al alcanzar una **tolerancia** predefinida, optimizando así el tiempo de cómputo frente a la complejidad del cálculo exacto.

Expansión de Laplace

Determinante por expansión de Laplace

Entrada: $A \in \mathcal{M}_{n \times n}(\mathbb{R})$

Salida: $\det(A) \in \mathbb{R}$

```

función DETERMINANTE( $A$ )
     $n \leftarrow \text{dimensión}(A)$ 
    si  $n = 1$  entonces
        | retornar  $A_{1,1}$ 
    si no si  $n = 2$  entonces
        | retornar  $A_{1,1}A_{2,2} - A_{1,2}A_{2,1}$ 
    fin si
     $\det \leftarrow 0$  (Inicialización del acumulador)
    para  $1 \leq j \leq n$  hacer (Expansión sobre la primera fila ( $i = 1$ ))
        |  $M \leftarrow \text{SUBMATRIZ}(A, \text{fila } 1, \text{columna } j)$ 
        |  $\text{signo} \leftarrow (-1)^{1+j}$ 
        |  $\det \leftarrow \det + (\text{signo} \cdot A_{1,j} \cdot \text{DETERMINANTE}(M))$ 
    fin para
    retornar  $\det$ 
fin función

```

Algoritmo

1.6.1

Ejercicio 1.6.1 (Implementación del Determinante y Análisis de Complejidad.): Programe en Python la función recursiva para el cálculo del determinante siguiendo el Algoritmo 1.6.1. Para obtener la submatriz M , puede utilizar el “slicing” de listas de Python.

Nota importante: Observe que para una matriz de $n \times n$, este algoritmo realiza aproximadamente $n!$ operaciones. Pruebe su código con matrices de tamaño 3×3 , 5×5 y 10×10 . ¿Nota un incremento desproporcionado en el tiempo de ejecución?

La limitación computacional que acabamos de experimentar con el determinante de Laplace no es un caso aislado. En el cómputo científico, la traducción directa de una fórmula matemática a código no siempre resulta en el algoritmo más eficiente. En la siguiente serie de problemas pondremos en práctica los conceptos desarrollados en este capítulo; no solo abordaremos demostraciones formales y la clasificación de EDPs, sino que continuaremos explorando cómo explotar las propiedades algebraicas de matrices y vectores para optimizar drásticamente nuestros programas.

Problemas 1.7

Para las demostraciones teóricas (Problemas 1, 2 y 3), repase los axiomas de espacio vectorial de la Sección 1.1 y las propiedades de las transformaciones lineales. Para los ejercicios computacionales, apóyese en los algoritmos desarrollados en el texto; el uso de la librería NumPy en Python le será de gran utilidad para comparar sus códigos y algoritmos desarrollados en este capítulo.

1. Dado $n \in \mathbb{N}$, considere $\mathbb{R}^n$, con la suma por componentes y el producto escalar por componentes. Demuestre que $\mathbb{R}^n$ con estas operaciones es un espacio vectorial real.
2. Sean $u, v, w \in \mathbb{R}^n$ vectores y $\lambda \in \mathbb{R}$ un escalar. Realice un código y una prueba en la ventana de comandos para cada una de las siguientes funciones:
 - a) $f(v, \lambda) := \lambda v$.
 - b) $f(u, v) := v + u$.
 - c) $f(u, v) := \langle v, u \rangle$.
 - d) $f(u, v, w, \lambda) := \langle u + \lambda v, w \rangle$.
3. Dado $n \in \mathbb{N}$, considere $\mathcal{M}_n(\mathbb{R})$, con la suma por componentes y el producto escalar por componentes. Demuestre que $\mathcal{M}_n(\mathbb{R})$ con estas operaciones es un espacio vectorial real.
4. Demuestre que toda matriz $A \in \mathcal{M}_n(\mathbb{R})$ es una transformación lineal sobre $\mathbb{R}^n$.

5. Sean $u, v \in \mathbb{R}^n$ vectores y $A, B \in \mathcal{M}_n(\mathbb{R})$ matrices cuadradas en los reales. Realice un pseudocódigo, un código y una prueba en la ventana de comandos para cada una de las siguientes funciones:

$$\text{a)} f(A, v) := Av. \quad \text{b)} f(A, B) := AB. \quad \text{c)} f(A, u, v) := \langle Au, v \rangle.$$

$$\text{d)} f(A, v) := \langle Av, v \rangle. \quad \text{e)} f(A, B, u, v) := ABu + ABv.$$

¿Observa que al menos hay tres formas de programar esta función? Recuerde que $AB(u + v) = A(Bu + Bv) = ABu + ABv$ ¿habrá alguna que sea **mejor** entre todas ellas?

6. Considere a $u, v, w \in \mathbb{R}^3$ vectores. Realice un pseudocódigo, un código y un test en la ventana de comandos para cada una de las siguientes funciones:

$$\text{a)} f(u, v) := v \times u.$$

$$\text{b)} f(u, v, w) := \begin{cases} \text{True}, & \text{si } u \perp (v \times w) \\ \text{False}, & \text{caso contrario} \end{cases}.$$

$$\text{c)} f(A) := \det(A), \quad \text{con } A \in \mathcal{M}_2(\mathbb{R}). \quad \text{d)} f(A) := \det(A), \quad \text{con } A \in \mathcal{M}_n(\mathbb{R}).$$

$$\text{e)} f(A, B) := \det(A, B), \quad \text{con } A, B \in \mathcal{M}_n(\mathbb{R}).$$

Para este ejercicio, recuerde que $\det(AB) = \det(A)\det(B)$. Por lo tanto, hay al menos dos formas de realizar este programa ¿Cuál será la **mejor** opción?

7. Para las siguientes EDPs, escriba el operador diferencial y un posible dominio de dicho operador, y determine si es lineal.

a) Ecuación de calor:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial^2 x} + f(x) \quad \text{con } \alpha \in \mathbb{R} \quad \text{y } f \in C([0, 1]).$$

b) Ecuación de onda:

$$\frac{\partial^2 u}{\partial^2 t} - \lambda^2 \frac{\partial^2 u}{\partial^2 x} = 0, \quad \text{con } \lambda \in \mathbb{R}.$$

c) Ecuación de Laplace:

$$\frac{\partial^2 u}{\partial^2 x} + \frac{\partial^2 u}{\partial^2 y} = 0.$$

d) Ecuación de Advección:

$$\frac{\partial u}{\partial x} + c \frac{\partial u}{\partial y} = 0 \quad \text{con } c > 0.$$

e) Ecuación KdV (Korteweg–De Vries):

$$\frac{\partial u}{\partial t} + u \frac{\partial^2 u}{\partial^2 x} + \frac{\partial^3 u}{\partial^3 x} = 0.$$

8. Para las EDPs del problema anterior, determine el grado, el orden, si son homogéneas, lineales, cuasilineales.
9. Investigue y describa un ejemplo físico o financiero de cada uno de los siguientes ítems, incluya, si existe, alguna ecuación diferencial parcial asociada al ejemplo.

- | | |
|--|---|
| a) Un sistema en estado estacionario de una variable espacial. | b) Un sistema en estado estacionario de dos variables espaciales. |
| c) Un sistema transitorio de tres variables espaciales. | d) Un sistema transitorio periódico de una variable espacial. |
-

Capítulo 2

ECUACIONES DIFERENCIALES ORDINARIAS

Contenido

2.1	Fundamentos	19
2.2	Discretización y métodos numéricos	24

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

2.1 Fundamentos

El paso del álgebra lineal al análisis diferencial representa la extensión natural de nuestro marco teórico: si en el capítulo anterior estudiamos transformaciones sobre espacios vectoriales de dimensión finita, ahora nuestro interés recae en operadores que actúan sobre espacios de funciones continuas.

2.1.1 Definición

Analíticamente, una ecuación diferencial surge como la relación intrínseca entre una función y sus variaciones locales. Geométricamente, plantea el problema fundamental de encontrar una curva (o variedad) cuya tangente en cada punto satisfaga una condición prescrita por un campo direccional dando paso a una de las estructuras más ricas del análisis matemático:

Definición 2.1.1 (Ecuación Diferencial): Se le denomina **ecuación diferencial** a una ecuación que relaciona una variable independiente (usualmente x), la función incógnita $y = y(x)$ que depende de dicha variable y las derivadas de la función $(y', y'', y^{(3)}, \dots, y^{(n)})$.

Formalmente, una ecuación diferencial (en adelante **ED**) se expresa de la siguiente forma:

$$F(x, y, y', y'', y^{(3)}, \dots, y^{(n)}) = 0.$$

Si la función incógnita $y = y(x)$ depende de una sola variable independiente x , se le denomina **ecuación diferencial ordinaria (EDO)**.

Ecuación Diferencial

Ecuación Diferencial Ordinaria

Para materializar esta abstracción formal y adquirir soltura en la lectura de sus operadores, resulta instructivo analizar la anatomía algebraica de distintos modelos matemáticos, contrastando explícitamente los casos ordinarios frente a los de varias dimensiones.

Ejemplo 2.1.1: Las siguientes expresiones son ejemplos clásicos de ecuaciones diferenciales ordinarias:

$$\text{a) } \frac{dy}{dx} + 5y = e^x. \quad \text{b) } y'' - 2y' + y = 0. \quad \text{c) } x \frac{d^3 y}{dx^3} - \left(\frac{dy}{dx} \right)^4 = \cos(x).$$

En todos los casos resulta evidente que la función incógnita y y sus derivadas dependen exclusivamente de una sola variable independiente (x). Nótese además que es una práctica estándar en la literatura alternar entre la notación de Leibniz $\left(\frac{dy}{dx} \right)$ y la notación de Lagrange (y').

Por el contrario, la siguiente expresión **no** es una EDO:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0.$$

Dado que la función incógnita $u = u(x, y)$ depende de dos variables independientes espaciales, esta recae en la categoría de una EDP, las cuales exigen operadores de derivación parcial (∂).

Definir de manera formal una ecuación diferencial es apenas el punto de partida. Para determinar qué herramientas analíticas o métodos numéricos serán viables en su resolución, es imperativo clasificar su estructura. La característica más restrictiva de estas ecuaciones es el nivel máximo de derivación que alcanzan, lo cual formalizamos a continuación.

Definición 2.1.2 (Orden de una ED): El **orden de una ecuación diferencial** se establece con la derivada de mayor orden de la función incógnita.

Orden de una Ecuación Diferencial

Para ilustrar esta métrica analítica y evitar la confusión frecuente entre los operadores diferenciales y las potencias algebraicas, analicemos el orden de las ecuaciones presentadas en el ejemplo anterior, añadiendo un caso de orden superior:

Ejemplo 2.1.2: Considere las siguientes ecuaciones diferenciales:

$$\text{a) } \frac{dy}{dx} + 5y = e^x. \quad \text{b) } y'' - 2y' + y = 0. \quad \text{c) } \left(\frac{d^2 y}{dx^2} \right)^3 + x \frac{d^4 y}{dx^4} = 0.$$

Análisis:

- La ecuación **a)** es de **primer orden**, ya que la derivada más alta aplicada sobre la función incógnita es la primera $\left(\frac{dy}{dx}\right)$.
- La ecuación **b)** es de **segundo orden**, dictado por el término y'' .
- La ecuación **c)** es de **cuarto orden**. Es de vital importancia no confundir la potencia algebraica (el cubo) con el operador diferencial; aunque la segunda derivada esté elevada a la tercera potencia, la derivada de mayor jerarquía presente en toda la expresión es $\frac{d^4y}{dx^4}$.

Ahora, considere la siguiente ED:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0.$$

Análisis: La ecuación es una EDP de **segundo orden**, puesto que las derivadas parciales más altas que se aplican sobre la función espacial $u(x, y)$ son de orden dos.

Con la ecuación correctamente clasificada, el objetivo natural recae en su resolución. A diferencia de los sistemas algebraicos lineales $Ax = b$, donde la incógnita suele ser un vector estático y único, en el cálculo diferencial nuestra incógnita es una función entera que debe preservar su validez analítica al ser sometida al operador diferencial.

Solución de una Ecuación Diferencial

Definición 2.1.3 (Solución de una ED): Se define como **solución de una ecuación diferencial** de orden n a toda función $y = \phi(x)$ que posea al menos n derivadas en un intervalo $I = (a, b)$ y que, al ser sustituida en la ecuación, la satisfaga plenamente. Es decir, que al reemplazar la función y sus derivadas, la ecuación se reduzca a una identidad verdadera para todo valor de x dentro de dicho intervalo.

La inversión de un operador diferencial introduce de manera natural grados de libertad analíticos en forma de constantes de integración. Esto implica que la ecuación diferencial no define una curva aislada, sino todo un espectro topológico de trayectorias. Para dotar de rigor a esta multiplicidad, la teoría clasifica las soluciones según su dependencia paramétrica.

Solución general

Definición 2.1.4 (Solución General): Se denomina **solución general** de una ecuación diferencial ordinaria de orden n a una familia de funciones que depende de n constantes arbitrarias (o parámetros), digamos $c_1, c_2, \dots, c_n$. Esta solución representa todas las posibles trayectorias que satisfacen la ecuación diferencial y suele expresarse de la siguiente forma:

$$G(x, y, c_1, \dots, c_n) = 0.$$

Para visualizar la naturaleza de esta familia de funciones y comprobar analíticamente cómo un mismo operador diferencial es capaz de gobernar toda una multiplicidad de trayectorias, analicemos el siguiente caso fundamental:

Ejemplo 2.1.3: Considere la ecuación diferencial de primer orden:

$$y' = y.$$

Comprobemos que la función $y(x) = e^x$ es una solución válida en todo $\mathbb{R}$. Al derivar la función propuesta obtenemos $y' = e^x$. Sustituyendo directamente en la ecuación diferencial, llegamos a la identidad analítica $e^x = e^x$, la cual es verdadera para cualquier valor de x .

Geoméricamente, esta solución representa una única curva de crecimiento exponencial en el plano cartesiano. Sin embargo, note que si proponemos funciones como $y_1(x) = 3e^x$ o $y_2(x) = -2e^x$, ambas satisfacen igualmente la ecuación diferencial ($3e^x = 3e^x$ y $-2e^x = -2e^x$). Esto evidencia que un mismo operador puede admitir múltiples curvas de forma simultánea, conformando un espectro de trayectorias como el que se ilustra en la Figura 2.1.

Si bien la solución general captura la dinámica global del modelo mediante una infinidad de curvas, el estudio de un fenómeno específico exige aislar una trayectoria determinista. Este proceso de selección geométrica se logra al colapsar los grados de libertad del sistema.

Definición 2.1.5 (Solución particular): Una **solución particular** es aquella que se deduce de la solución general al asignarle valores numéricos específicos a sus constantes arbitrarias. En las

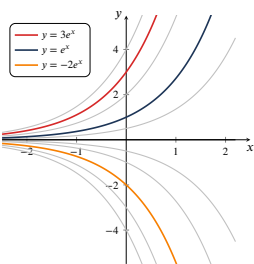


Figura 2.1: Espectro de curvas integrales de la forma $y = Ce^x$. Cada curva individual es una solución geométrica válida para la ecuación $y' = y$.

Solución particular

aplicaciones, estos parámetros se fijan de manera única al evaluar la ecuación en las condiciones iniciales o de frontera del problema.

Para ilustrar este colapso paramétrico con una geometría distinta, consideremos una nueva familia de trayectorias y apliquemos una restricción de estado:

Ejemplo 2.1.4: Considere la ecuación diferencial de primer orden:

$$y' = 2x.$$

Por integración directa, es evidente que su solución general corresponde a la familia de parábolas $y(x) = x^2 + C$.

Supongamos que las restricciones del problema exigen aislar la trayectoria que cruza exactamente por el punto $(1, 2)$. Para hallar el valor de la constante arbitraria C , evaluamos esta condición en nuestra solución general:

$$2 = (1)^2 + C \implies C = 1.$$

Al sustituir este parámetro, el espectro infinito colapsa en la **solución particular**:

$$y(x) = x^2 + 1.$$

Como se ilustra en la Figura 2.2, la resolución de la ecuación diferencial puede entenderse como un proceso de filtrado geométrico en tres fases: primero, el operador establece un campo direccional; segundo, la integración revela un catálogo infinito de traslaciones verticales compatibles con dicho campo; y finalmente, la evaluación en una condición inicial específica aísla una única curva determinista.

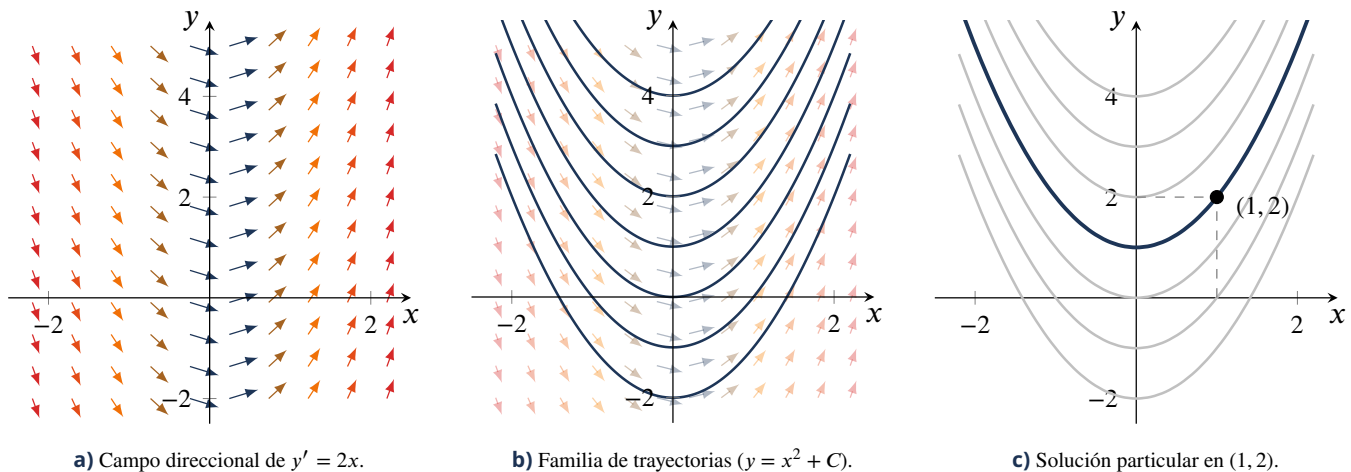


Figura 2.2: Evolución geométrica de la resolución de una ED: subfigura **a)** el operador define las pendientes locales en cada punto del plano; subfigura **b)** la integración revela el espectro total de funciones compatibles con el campo; subfigura **c)** la imposición de una condición inicial colapsa la incertidumbre en una única trayectoria determinista.

Resulta tentador asumir que toda trayectoria matemáticamente viable se encuentra contenida en el catálogo paramétrico que ofrece la solución general. No obstante, el análisis de ecuaciones diferenciales no lineales revela la existencia de curvas anómalas que, pese a satisfacer rigurosamente al operador diferencial, escapan por completo a la formulación de la familia principal.

Definición 2.1.6 (Solución singular): Se le llama **solución singular** a una función que satisface plenamente la ecuación diferencial, pero que es imposible de obtener a partir de la solución general sin importar qué valor se le asigne a las constantes. Geométricamente, las soluciones singulares suelen representar curvas envolventes de la familia paramétrica descrita por la solución general.

Solución singular

Para materializar este concepto, analizaremos una de las estructuras no lineales más célebres del cálculo: la ecuación de Clairaut.

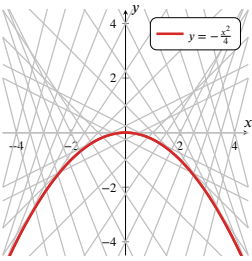


Figura 2.3: La parábola $y = -x^2/4$ es la solución singular de la ED. Actúa como la envolvente de la familia de soluciones generales $y = Cx + C^2$ (rectas grises).

Ejemplo 2.1.5: Considere la siguiente ecuación diferencial no lineal de primer orden:

$$y = xy' + (y')^2.$$

Al resolver esta ecuación —mediante un cambio de variable acoplado a una derivación total—, se obtiene una solución general conformada por una familia infinita de líneas rectas:

$$y(x) = Cx + C^2.$$

Si verificamos esta familia, notaremos que $y' = C$. Al sustituir en la ED original, obtenemos $Cx + C^2 = x(C) + C^2$, lo cual es una identidad perfecta para cualquier valor real de C .

Sin embargo, existe una función adicional que satisface esta misma ecuación:

$$y(x) = -\frac{x^2}{4}.$$

Derivando esta nueva propuesta, tenemos $y' = -x/2$. Sustituyendo en la ED original:

$$-\frac{x^2}{4} = x\left(-\frac{x}{2}\right) + \left(-\frac{x}{2}\right)^2 = -\frac{x^2}{2} + \frac{x^2}{4} = -\frac{x^2}{4}.$$

La identidad se cumple de manera impecable. Sin embargo, no existe ningún número real que podamos asignarle a la constante C en la familia de rectas ($y = Cx + C^2$) para que esta se transforme milagrosamente en una parábola ($y = -x^2/4$).

Como ilustra la Figura 2.3, esta **solución singular** es la curva envolvente del sistema: un límite geométrico que es tangente a todas y cada una de las soluciones generales, naciendo del “caos” de las rectas superpuestas.

Comprender la distinción geométrica y analítica entre estas tres categorías es el núcleo del análisis diferencial. Mientras que la solución general nos brinda el catálogo completo de curvas que absorben la dinámica de la ecuación, la resolución de un problema específico exige determinismo. Geométricamente, la familia paramétrica llena una región del plano; aislar una única *curva integral* (es decir, la solución particular) requiere fijar coordenadas de anclaje. Para lograrlo, es indispensable someter la ecuación a restricciones algebraicas externas, dando paso a los esquemas de formalización de condiciones.

2.1.2 Problemas de condiciones iniciales

El esquema de restricción más directo, y el estándar por excelencia en el estudio de sistemas que evolucionan de forma paramétrica, es aquel que prescribe el estado geométrico de la curva en un único punto de partida. Esto último se aborda en la siguiente:

Definición 2.1.7 (Problema de condiciones iniciales): Un **problema de condiciones iniciales** (o de valor inicial o problema de Cauchy) consiste en una ecuación diferencial sujeta a condiciones adicionales que deben ser satisfechas en un **único punto** de la variable independiente. Para una ecuación de orden n , se requieren n condiciones iniciales, formuladas típicamente como:

$$y(x_0) = y_0, \quad y'(x_0) = y_1, \quad \dots, \quad y^{(n-1)}(x_0) = y_{n-1}.$$

A estas restricciones algebraicas se les conoce como **condiciones iniciales**. Una de las aplicaciones más inmediatas es modelar el estado de un sistema en un instante de tiempo o posición inicial conocidos.

Para ilustrar la mecánica de este esquema de restricción, analicemos un caso unidimensional y, posteriormente, un sistema mecánico que demanda múltiples condiciones:

Ejemplo 2.1.6 (PCI de primer orden): Considere la siguiente ecuación diferencial ordinaria sujeta a una condición inicial evaluada en el origen:

$$y' + 2xy = 0, \quad y(0) = 3.$$

Por métodos de separación de variables, se determina que la solución general es la familia paramétrica

Problema de Condiciones Iniciales

$y(x) = Ce^{-x^2}$. Para encontrar la trayectoria particular que resuelve este **PCI**, exigimos que la función cumpla la restricción geométrica $y(0) = 3$:

$$3 = Ce^{-(0)^2} \implies C = 3.$$

Por lo tanto, la única curva que satisface tanto la dinámica del operador como el anclaje inicial es $y(x) = 3e^{-x^2}$.

Ejemplo 2.1.7 (PCI de segundo orden): La segunda ley de Newton para un objeto en caída libre cerca de la superficie terrestre se modela mediante una ecuación diferencial de segundo orden, donde la variable independiente es el tiempo t :

$$\frac{d^2y}{dt^2} = -g.$$

La integración sucesiva de esta aceleración constante arroja la solución general:

$$y(t) = -\frac{1}{2}gt^2 + C_1t + C_2.$$

Como la ecuación es de orden $n = 2$, el problema de Cauchy exige evaluar 2 condiciones iniciales en el mismo instante temporal (usualmente $t = 0$) para definir el sistema de forma determinista:

- **Posición inicial:** $y(0) = y_0 \implies C_2 = y_0$.
- **Velocidad inicial:** $y'(0) = v_0 \implies C_1 = v_0$.

El colapso de los grados de libertad revela la conocida ecuación de movimiento parabólico:

$$y(t) = -\frac{1}{2}gt^2 + v_0t + y_0.$$

El enfoque de un problema de Cauchy es intrínsecamente propagativo: ancla la dinámica geométrica en un punto de referencia x_0 y permite que el operador diferencial dicte la evolución de la curva a lo largo del dominio. Si bien esta estructura analítica es ideal para sistemas causales (donde el estado inicial determina por completo el futuro), el análisis de ciertos modelos requiere que la trayectoria se ajuste de forma simultánea a restricciones impuestas en los extremos de su dominio topológico.

2.1.3 Problemas con valores en la frontera

Mientras que los problemas de valor inicial fijan un nodo del cual emana la trayectoria (típicamente asociado a la evolución paramétrica de una variable como el tiempo), el análisis de medios continuos y operadores espaciales requiere imponer restricciones en las fronteras de la región de validez.

Definición 2.1.8 (Problema de valores en la frontera): A diferencia de la Definición 2.1.7, un **problema con valores en la frontera (PVF)** exige que la solución de la ecuación diferencial satisfaga condiciones especificadas en **distintos puntos** del dominio, generalmente en los extremos de un intervalo $[a, b]$. Para una ecuación de segundo orden, las **condiciones de frontera** más comunes toman la forma:

$$y(a) = \alpha, \quad y(b) = \beta.$$

A diferencia de las condiciones iniciales, que garantizan una trayectoria determinista al “empujar” la curva desde un solo punto, fijar la curva en dos extremos distintos introduce una rigidez geométrica que puede fracturar el sistema. Para ilustrar esta sensibilidad, analicemos el comportamiento de un mismo operador bajo dos juegos de fronteras distintos:

Ejemplo 2.1.8 (PVF con solución única): Considere el modelo de deformación elástica gobernado por $y'' + y = 0$. Su solución general es la combinación lineal:

$$y(x) = C_1 \cos(x) + C_2 \sin(x).$$

Si exigimos que la curva esté anclada en el origen $y(0) = 0$ y alcance una altura específica en otro punto $y(\pi/2) = 2$, evaluamos los extremos:

Problema con Valores en la Frontera

- Frontera izquierda: $y(0) = C_1 \cos(0) + C_2 \sin(0) \implies C_1 = 0$.
- Frontera derecha: $y(\pi/2) = 0 \cdot \cos(\pi/2) + C_2 \sin(\pi/2) \implies C_2 = 2$.

El sistema colapsa en una única solución perfectamente definida: $y(x) = 2 \sin(x)$.

Ejemplo 2.1.9 (PVF sin solución): Tomando el mismo operador y solución general $y(x) = C_1 \cos(x) + C_2 \sin(x)$, modifiquemos la frontera derecha para exigir que la curva pase por $y(\pi) = 1$:

- Frontera izquierda: $y(0) = C_1 = 0 \implies y(x) = C_2 \sin(x)$.
- Frontera derecha: $y(\pi) = C_2 \sin(\pi) = C_2(0) = 0$.

Llegamos a la evidente contradicción $0 = 1$. Físicamente, la oscilación espacial inherente del seno le prohíbe cruzar por el punto $(\pi, 1)$ si partió del origen. Este problema **carece de solución**.

Establecer la ecuación diferencial y prescribir sus restricciones geométricas —ya sean iniciales o de frontera— constituye apenas el planteamiento teórico del modelo. Sin embargo, desde una perspectiva estrictamente teórica, la simple formulación matemática no garantiza que el sistema posea un comportamiento real y bien definido; es decir, podría no existir ninguna curva que satisfaga simultáneamente el operador diferencial y las restricciones impuestas, o bien, podrían existir múltiples trayectorias que lo hagan.

2.1.4 Teorema de existencia y unicidad

Tanto en el desarrollo analítico como en la futura implementación de algoritmos numéricos en Python, plantear la ecuación y sus restricciones constituye únicamente la formulación del modelo. Antes de invertir recursos computacionales para aproximar una solución, el rigor matemático exige certificar que el problema está bien planteado; es decir, debemos demostrar categóricamente que la solución existe y que no hay ambigüedad en su comportamiento. Para este propósito,

Teorema 2.1.1 (Existencia y unicidad de la solución): Sea $y' = f(x, y)$ una ecuación diferencial donde la función $f(x, y)$ está definida en un conjunto A que contiene al punto (x_0, y_0) . Si la función satisface las siguientes condiciones:

- $f(x, y)$ es continua en el conjunto A ,
- $f(x, y)$ admite derivada parcial $\frac{\partial f}{\partial y}$, la cual es continua en el conjunto A .

Entonces existe una y sólo una solución $y = \psi(x)$ de la ecuación dada que satisface la condición inicial $y(x_0) = y_0$.

Note que el Teorema 2.1.1 nos dice cuáles son las condiciones **suficientes** para la existencia de la solución única del problema de Cauchy, sin embargo, dichas condiciones no son **necesarias**. Esto quiere decir que puede existir una solución única a pesar de que esta no cumpla con una o ninguna de las condiciones simultáneamente.

2.2 Discretización y métodos numéricos

La imposibilidad de encontrar soluciones analíticas cerradas para la gran mayoría de las ecuaciones diferenciales no lineales exige un cambio de paradigma: abandonar la búsqueda de una función continua exacta y conformarnos con una aproximación discreta. Este proceso, conocido como **discretización**, consiste en evaluar la solución únicamente en un conjunto finito de puntos ordenados dentro del dominio, transformando el problema del cálculo diferencial en un proceso iterativo algebraico.

2.2.1 Método de Euler

El esquema numérico más elemental para resolver un problema de valor inicial fue propuesto por Leonhard Euler. Su deducción natural proviene de truncar la expansión en series de Taylor de la función solución. Supongamos que $y(x)$ es la solución exacta del problema de valor inicial $y' = f(x, y)$ con $y(x_0) = y_0$. Si expandimos $y(x)$ alrededor de un punto x_j con un incremento pequeño h , obtenemos:

$$y(x_j + h) = y(x_j) + hy'(x_j) + \frac{h^2}{2!} y''(x_j) + \mathcal{O}(h^2).$$

Al truncar la serie después del término lineal y sustituir la dinámica impuesta por la ecuación diferencial ($y'(x_j) = f(x_j, y_j)$), llegamos a la regla de iteración fundamental.

Definición 2.2.1 (Método de Euler): Dado un problema de valor inicial $y' = f(x, y)$ con $y(x_0) = y_0$, el **método de Euler** construye una solución aproximada mediante una sucesión de puntos (x_j, y_j) . Definiendo un tamaño de paso constante $h = x_{j+1} - x_j$, la iteración está dada por:

$$y_{j+1} = y_j + hf(x_j, y_j), \quad \text{para } j = 0, 1, 2, \dots, n-1.$$

Geométricamente, el método aproxima la curva integral avanzando pequeños pasos de longitud h a lo largo de la recta tangente evaluada en el punto actual.

Como se ilustra en la Figura 2.4, esta linealización geométrica conlleva un costo analítico: al sustituir la verdadera curvatura de la solución por un segmento recto, se introduce una discrepancia ineludible en cada avance, conocida como *error de truncamiento local*.

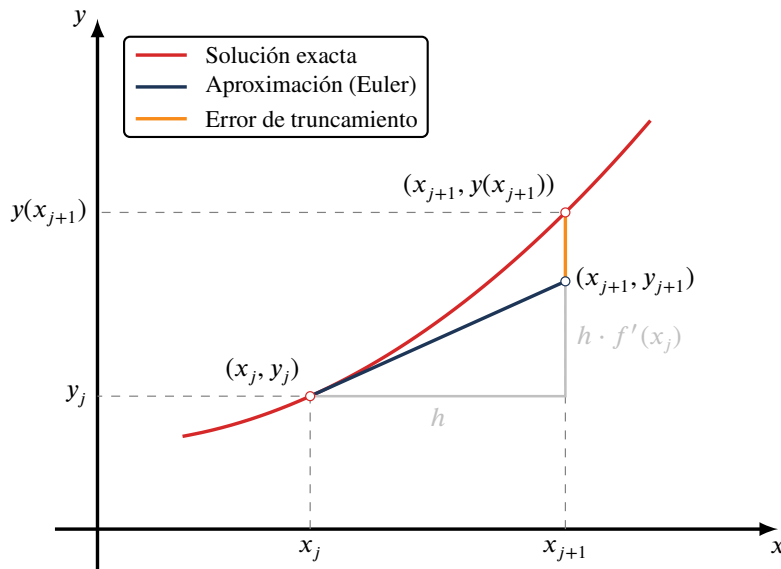


Figura 2.4: Interpretación geométrica del Método de Euler. La solución numérica (x_{j+1}, y_{j+1}) se obtiene avanzando un paso de longitud h sobre la recta tangente a la condición inicial (x_j, y_j) . La convexidad de la curva real genera una divergencia conocida como *error de truncamiento local*.

A pesar de esta inherente limitación teórica, la simplicidad de su formulación hace que, a nivel computacional, el método de Euler sea el arquetipo perfecto de un algoritmo de marcha directa (explícito). Conociendo el estado inicial, basta con calcular la pendiente local, dar un paso hacia el futuro para estimar el nuevo estado y repetir el proceso hasta alcanzar el límite del dominio deseado $[a, b]$.

Método de Euler

Entrada: Función $f(x, y)$, intervalo $[a, b]$, condición inicial y_a , número de pasos $n \in \mathbb{N}$

Salida: Vectores de discretización espaciales x y aproximaciones y

función EULER(f, a, b, y_a, n)

$h \leftarrow \frac{b-a}{n}$ (Cálculo del tamaño de paso)

$x \leftarrow \text{vector.ceros}(n+1)$

$y \leftarrow \text{vector.ceros}(n+1)$

$x_0 \leftarrow a$ (Anclaje espacial)

$y_0 \leftarrow y_a$ (Condición inicial)

para $0 \leq j \leq n-1$ **hacer**

Algoritmo

2.2.1

```

    |    $x_{j+1} \leftarrow x_j + h$  (Avance en el dominio)
    |    $y_{j+1} \leftarrow y_j + h \cdot f(x_j, y_j)$  (Proyección sobre la tangente)
    |   fin para
    |   retornar  $(x, y)$ 
    |   fin función

```

2.2.2 Runge-Kutta de cuarto orden (RK-4)

Como evidenciamos en el análisis geométrico del método de Euler, aproximar una trayectoria curva mediante un segmento recto evaluado únicamente en el extremo izquierdo del intervalo genera un error de truncamiento sustancial $\mathcal{O}(h^2)$. Para mejorar la precisión sin incurrir en el costoso cálculo de derivadas analíticas de orden superior, los matemáticos Carl Runge y Martin Kutta desarrollaron una familia de métodos iterativos que exploran y promedian estratégicamente las pendientes en puntos intermedios del intervalo.

El miembro más célebre de esta familia es el método de cuarto orden (comúnmente abreviado como RK4). Por su excepcional balance entre precisión computacional y costo operativo, es considerado el “caballo de batalla” en la resolución numérica de ecuaciones diferenciales ordinarias.

Método RK-4

Definición 2.2.2 (Método de Runge-Kutta de cuarto orden (RK-4)): Dado un problema de valor inicial $y' = f(x, y)$ con $y(x_0) = y_0$, el **método RK-4** construye una solución aproximada calculando cuatro pendientes de prueba en cada subintervalo de longitud h :

$$\begin{aligned}
 k_1 &= f(x_j, y_j), \\
 k_2 &= f\left(x_j + \frac{h}{2}, y_j + \frac{h}{2}k_1\right), \\
 k_3 &= f\left(x_j + \frac{h}{2}, y_j + \frac{h}{2}k_2\right), \\
 k_4 &= f(x_j + h, y_j + hk_3).
 \end{aligned}$$

La iteración fundamental proyecta el nuevo estado ponderando estas cuatro pendientes, otorgando mayor peso estadístico a las evaluaciones centrales para anular los errores de curvatura:

$$y_{j+1} = y_j + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4), \quad \text{para } j = 0, 1, 2, \dots, n-1.$$

A nivel computacional, el algoritmo RK4 mantiene la misma estructura de marcha explícita que Euler, pero enriquece drásticamente la fase de avance. Dentro de cada iteración, el algoritmo realiza una exploración prospectiva: utiliza k_1 para dar un medio paso de prueba y calcular k_2 , usa k_2 para refinar ese medio paso calculando k_3 , y finalmente lanza un paso completo con k_3 para evaluar la pendiente final k_4 . Solo entonces se compromete a actualizar el estado del sistema.

Algoritmo

2.2.2

Runge-Kutta de cuarto orden (RK-4)

Entrada: Función $f(x, y)$, intervalo $[a, b]$, condición inicial y_a , número de pasos $n \in \mathbb{N}$

Salida: Vectores de discretización espaciales x y aproximaciones y

función RK4(f, a, b, y_a, n)

```

    |    $h \leftarrow \frac{b-a}{n}$  (Cálculo del tamaño de paso)

```

```

    |    $x \leftarrow \text{vector.ceros}(n+1)$ 

```

```

    |    $y \leftarrow \text{vector.ceros}(n+1)$ 

```

```

    |    $x_0 \leftarrow a$  (Anclaje espacial)

```

```

    |    $y_0 \leftarrow y_a$  (Condición inicial)

```

```

    |   para  $0 \leq j \leq n-1$  hacer

```

```

    |       (Cálculo de las pendientes ponderadas)

```

```

    |        $k_1 \leftarrow f(x_j, y_j)$ 

```

```

     $k_2 \leftarrow f\left(x_j + \frac{h}{2}, y_j + \frac{h}{2} \cdot k_1\right)$ 
     $k_3 \leftarrow f\left(x_j + \frac{h}{2}, y_j + \frac{h}{2} \cdot k_2\right)$ 
     $k_4 \leftarrow f(x_j + h, y_j + h \cdot k_3)$ 

     $x_{j+1} \leftarrow x_j + h$  (Avance en el dominio)
     $y_{j+1} \leftarrow y_j + \frac{h}{6} \cdot (k_1 + 2k_2 + 2k_3 + k_4)$  (Avance ponderado)
  fin para
  retornar  $(x, y)$ 
fin función

```

2.2.3 Diferencias finitas

A diferencia de los métodos evaluados en problemas de valor inicial, donde la solución avanza paso a paso en el tiempo, la resolución de problemas con valores en la frontera exige analizar el dominio en su totalidad de forma simultánea. Para lograrlo, el método de diferencias finitas se apoya en un concepto geométrico fundamental: la discretización espacial.

El primer paso ineludible consiste en abandonar el continuo. Matemáticamente, esto significa sustituir el intervalo ininterrumpido $[a, b]$ donde está definido el problema por una cuadrícula o esqueleto finito de puntos interconectados.

Definición 2.2.3 (Malla unidimensional): Sea un dominio espacial definido por el intervalo continuo $x \in [a, b]$. Una **malla** (o red) se construye particionando este intervalo en n subintervalos de igual longitud. La separación constante entre cada punto, conocida como **tamaño de paso espacial**, se define como:

$$h = \frac{b - a}{n}.$$

Esta partición genera un conjunto ordenado de $n + 1$ **nodos discretos** definidos por la progresión aritmética:

$$x_i = a + i \cdot h, \quad \text{para } i = 0, 1, 2, \dots, n.$$

Al imponer esta estructura geométrica sobre el problema, la variable independiente pierde su naturaleza fluida. En consecuencia, la función incógnita $y(x)$ deja de ser una curva continua de infinitos puntos para colapsar en un vector de dimensión finita. Nuestro objetivo numérico se reduce entonces a encontrar el valor discreto $y_i = y(x_i)$ en cada nodo específico de la malla.

Una vez que el dominio ha sido discretizado en nodos (x_{i-1}, x_i, x_{i+1}) , necesitamos una herramienta que nos permita “leer” las derivadas continuas de la ecuación diferencial original utilizando únicamente la información de estos puntos discretos vecinos. Aquí es donde interviene el álgebra de series.

Definición 2.2.4 (Operadores de Diferencias Finitas): El método de diferencias finitas aproxima las derivadas de una función resolviendo el desarrollo en serie de Taylor truncado alrededor de un nodo x_i . Para un tamaño de paso $h > 0$ dictado por la malla, las expansiones laterales hacia los nodos adyacentes son:

$$y(x_{i+1}) = y(x_i) + hy'(x_i) + \frac{h^2}{2!}y''(x_i) + \mathcal{O}(h^3),$$

$$y(x_{i-1}) = y(x_i) - hy'(x_i) + \frac{h^2}{2!}y''(x_i) + \mathcal{O}(h^3).$$

A partir de estas expansiones, es posible despejar la primera derivada $y'(x_i)$ para obtener tres esquemas de medición sobre la malla:

- **Diferencia hacia adelante (Forward difference):** Despejando de la primera expansión, se

Malla espacial

obtiene un error de truncamiento $\mathcal{O}(h)$:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

- **Diferencia hacia atrás (Backward difference):** Empleando la expansión hacia atrás, se mantiene un esquema de orden $\mathcal{O}(h)$:

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}$$

- **Diferencia central (Central difference):** Al restar la serie de $f(x-h)$ a la de $f(x+h)$, los términos pares se anulan. Esto produce un esquema simétrico con un error sustancialmente menor, de orden $\mathcal{O}(h^2)$:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

La flexibilidad del operador de diferencias finitas permite extender la aproximación a derivadas de mayor orden. Sumando ambas expansiones de Taylor, los términos con derivadas impares se cancelan, permitiendo aislar $f''(x)$ para construir la **diferencia central de segundo orden**:

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

Al aplicar estos operadores discretos sobre una malla particionada $x_i = a + i \cdot h$, el problema continuo original se transforma en un sistema de ecuaciones algebraicas. En el caso lineal unidimensional, la interacción de cada nodo exclusivamente con sus vecinos inmediatos genera un sistema regido por una matriz tridiagonal, cuya resolución resulta altamente eficiente para una computadora.

Diferencia central de segundo orden

Algoritmo

2.2.3

Diferencias finitas en 1D (Problema de Poisson)

Entrada: Función fuente $g(x)$, intervalo $[a, b]$, condiciones de frontera y_a, y_b , subintervalos $n \in \mathbb{N}$

Salida: Vector espacial x y aproximaciones y para el PVF $y'' = g(x)$

función DIFFINITAS1D(g, a, b, y_a, y_b, n)

$h \leftarrow \frac{b-a}{n}$

$x \leftarrow \text{vector.ceros}(n+1)$

$y \leftarrow \text{vector.ceros}(n+1)$

$x_0 \leftarrow a, y_0 \leftarrow y_a$ (Frontera izquierda)

$x_n \leftarrow b, y_n \leftarrow y_b$ (Frontera derecha)

(Ensamblaje del sistema tridiagonal $Ay_{int} = b$)

$A \leftarrow \text{matriz.ceros}(n-1, n-1)$

$b \leftarrow \text{vector.ceros}(n-1)$

para $1 \leq i \leq n-1$ **hacer**

$x_i \leftarrow a + i \cdot h$

$A_{i,i} \leftarrow -2$ (Diagonal principal)

si $i > 1$ **entonces** $A_{i,i-1} \leftarrow 1$

fin si (Subdiagonal)

si $i < n-1$ **entonces** $A_{i,i+1} \leftarrow 1$

fin si (Superdiagonal)

$b_i \leftarrow h^2 \cdot g(x_i)$

fin para

(Ajuste de cargas por condiciones de frontera)

$b_1 \leftarrow b_1 - y_a$

$b_{n-1} \leftarrow b_{n-1} - y_b$

$y_{1:n-1} \leftarrow \text{resolver_sistema}(A, b)$ (Solución algebraica)

retornar (x, y)

fin función

Para visualizar por qué la diferencia central ofrece una convergencia de orden superior frente a las aproximaciones laterales, resulta instructivo abandonar temporalmente el rigor algebraico de las series de Taylor y analizar el comportamiento geométrico de estos operadores.

Aproximar una primera derivada mediante diferencias finitas equivale a sustituir la **recta tangente** exacta en un punto por una **recta secante** que conecta nodos adyacentes de la malla:

- Las **diferencias hacia adelante y hacia atrás** calculan la pendiente de las secantes asimétricas que unen $(x, f(x))$ con su vecino derecho o izquierdo, respectivamente. Al estar ancladas geométrica y dinámicamente hacia un solo lado, la secante ignora la información del dominio opuesto, generando un sesgo que se traduce en el error de truncamiento lineal $\mathcal{O}(h)$.
- La **diferencia central**, por el contrario, calcula la pendiente de la secante que cruza desde $(x - h, f(x - h))$ hasta $(x + h, f(x + h))$. En curvas de variación suave, esta secante que “salta” el nodo central resulta ser casi perfectamente paralela a la recta tangente en x , cancelando el error direccional y mejorando dramáticamente la precisión a $\mathcal{O}(h^2)$.

Por su parte, la **diferencia central de segundo orden** abandona la linealidad de las rectas secantes para adentrarse en la medición de concavidades. Geométricamente, evaluar la expresión computacional $\frac{f(x+h)-2f(x)+f(x-h)}{h^2}$ es exactamente equivalente a aislar la única parábola que interpola los tres puntos espaciales adyacentes $-(x-h, f_{i-1}), (x, f_i)$ y $(x+h, f_{i+1})$ — y calcular su segunda derivada constante. Es la métrica discreta perfecta para evaluar qué tanto la curva real se está desviando de ser una línea recta.

Bibliografía

- [1] Burden, R. L. y Faires, J. D. *Numerical Analysis*. 9.^a ed. (Cengage Learning, 2011).
- [2] Hoffman, K. y Kunze, R. *Álgebra Lineal*. 2.^a ed. (Prentice-Hall, 1973).
- [3] Krasnov, M., Kiselev, A. y Makarenko, G. *Problemas de ecuaciones diferenciales ordinarias*. 4.^a ed. (Moscú: Editorial Mir, 1984).
- [4] Zill, D. G. *Ecuaciones Diferenciales con aplicaciones de modelado*. 10.^a ed. (Cengage Learning, 2012).
- [5] Zill, D. G. *Ecuaciones Diferenciales con problemas con valores en la frontera*. 10.^a ed. (Cengage Learning, 2024).

Esta página fue dejada en blanco intencionalmente.

